TYPE DISCIPLINE FOR UNOBSERVABLE UPDATE

How a type discipline makes a write correct, and how that becomes a compiler optimisation on reference count one

TYPE DISCIPLINE FOR UNOBSERVABLE UPDATE

THE IDEA, IN ONE LINE

A pure language cannot mutate. But a compiler can update **in place** — without changing what a program **means**.

This deck walks one implication, in easy steps:

- **type discipline** → writes are **unobservable** → so an update is **correct**;
- a compiler realises it as an **optimisation on reference count one**;
- and a type discipline can **ensure** that count.

THE PROBLEM

Lean has no mutable state. `a.set! i v` is a function: it denotes a `new` array. Nothing in the source can change a value that already exists.

Copying is the price of that purity: a whole-object allocation for every "update".

We want the cheap thing — a mutation — without losing the pure meaning.

THE IDEA: MAKE THE WRITE UNOBSERVABLE

An update in place is **correct** exactly when it **cannot be observed**.

If the old cell has **no other reference**, then "the same object with one slot changed" and "a new object, copied" are **indistinguishable** — the same value.

So: a write is legal to mutate **exactly when it is unobservable**. That is the whole trick.

REALISED AS A COMPILER OPTIMISATION

The `Lean` compiler already does this. At every store it asks one question: `is this object's reference count exactly 1 here?`

> count `1` → the old value is dead: store into it, in place.
>
> count `2+` → something may still read the old value: allocate a copy.
>
> count `0` → a compile-time constant: never unique, always copy.

So the mechanism is `given`. `It` compiles away the rest `whenever it can see` the count is one.

SO THE REAL QUESTION IS: CAN WE ENSURE RC = 1?

The mechanism fires `if the count is 1`. The whole problem is now: `make the count 1`.

And that is a `static` property of the program — how many references the code holds — so a `type discipline` can decide it.

A type discipline that ensures RC = 1 ⇒ the compiler's in-place update is both `correct` (unobservable) and `real` (performed).

THE TYPE DISCIPLINE: OWNERSHIP

The discipline is **ownership**: a write goes through a cell the program holds **exactly once**.

A written name is used **once** in the scope that binds it, and is bound to a **construction** — the shapes where a **second holder** exists are **refused**: duplication, an alias, a use in a sibling position.

../LEAN/SCIDONIA/RC/OWNERSHIPTYPED/JUDGEMENT.LEAN

```
/-- A write is admitted only at the owner the subject names, on the subject's
own name and at index zero. -/
| write {Γ : TyEnv} (x : Var) {a : Expr} (hx : Γ x = some .cell)
    (ha : Obs Γ a slotObs) : Own Γ (.held x) (.store (.var x) 0 a) .cell
```

THE IMPLICATION, STEP BY STEP

`Typed program` ⇒ the write's cell is `uniquely held` (the ownership rule).

⇒ under a `correspondence`, the runtime `count is 1` (`bridge`).

⇒ so the write is `destructive` (`store_owned_is_destructive`).

⇒ and it is `unobservable`: the in-place run and the copy run are the `same value` (`evalArray_sound`).

../LEAN/SCIDONIA/RC/CORRESPONDENCE.LEAN

```
/-- The bridge: a store typed at `Γ`, whose environment `Γ` counts its subject once,
at a configuration corresponding to `(ρ, m)` in the sense of `Corr`, meets the target
write theorem's two premises --- the account holds the subject's occurrence (`Owns`),
and the account counts the reference the environment names for it exactly once. -/
theorem bridge {ρ : TEnv} {m : Multi} {Γ : Occs} {y : Var} {r : Ref} {i : Nat} {v : Expr}
    (h : Corr ρ m Γ) (hlook : ρ.lookup y = some r) (hone : Γ.count y = 1) :
    Owns ρ m (.store (.var y) i v) ∧ (expand m).count r = 1 := by
```

COUNT ONE ⇒ DESTRUCTIVE

When the account counts the subject's reference **exactly once** — and no heap slot, no run handle names the cell — the write keeps the cell's location.

A store at any other count **copies**: the written slots land at a fresh location.

../LEAN/SCIDONIA/RC/OWNERSHIP.LEAN

```
theorem store_owned_is_destructive {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi}
    {y : Var} {r : Ref} {i : Nat} {v : Expr} {ks : List Ref} (hc : Couple c hs ρ m)
    (hρ : ρ.lookup y = some r) (hown : Owns ρ m (.store (.var y) i v))
    (hm : (expand m).count r = 1) (hslot : (CHeap.slots c).count r = 0)
    (hrun : hs.count r = 0) (hks : r ∉ ks) :
    (update c (hs ++ expand m) r ks).1.lookup r = some ks := by
```

AND IT IS UNOBSERVABLE

The sealed evaluation and the copying reduction of the same term reach exactly the same value.

That is the payoff: the update is not just correct in intent — it is proved equal to the pure meaning.

../LEAN/SCIDONIA/RC/OWNERSHIPTYPED/REFINEMENT.LEAN

```
/-- The agreement: an admitted, closed program of the fragment evaluated by the
sealed runner reaches exactly the value the copying λPure reduction of the *same*
term reaches. The reduction hypothesis is a statement about the existing
`Scidonia.RC.StepStar`, and `evalArray` is the direct side's evaluation. -/
theorem evalArray_sound {o : ObsDesc} {e v : Expr}
    (h : Own (TyEnv.empty : TyEnv) .none e (.obs o)) (_hc : Closed e)
    (hs : StepStar e v) (hv : Value v) : evalArray e = some v := by
```

A TREE: MUCH SHARED FOR READS, ONLY THE WRITE TOTAL

The same discipline scales to trees: a read borrows a fraction of the spine's nodes — so many readers can share one spine — and a write holds the whole of its target subtree.

That is "much of the tree shared out for reads, and only the write has to be a total fraction."

../LEAN/SCIDONIA/RC/TREEOWNERSHIP/JUDGEMENT.LEAN

```
/-- The account admits a write by the holder `w` at a node: the holder has the
whole of every node of the target's subtree, and a positive share of every proper
ancestor of the target. The two are read at the same nodes by construction — one
`Writes` per node of the subtree, one positive `Reads` per proper ancestor — so
nothing is inferred from a fraction somewhere else. -/
def WriteHolds {d : Layout} (A : Account d) (w : Nat) (p : Path d) : Prop :=
  (∀ q, InSubtree p q → Writes A q) ∧ (∀ r, PrefixOf r p → r ≠ p → ∃ q, 0 < q ∧ Reads A r w
q)
```

THE HONEST BOUNDARY

This is a model claim, proved: the discipline yields count 1 in the counted heap, and the update is correct.

Not proved: that the model's count is the machine's `lean_is_exclusive` on a real object — that correspondence is the open link.

And Lean's reuse pass needs the update's shape: RC = 1 alone is not sufficient without it.

In-place update of compiled code is measured, not a theorem.

WHAT IS PROVED, AND WHAT IS NOT

Proved: writes unobservable (the two semantics agree); the correspondence from ownership to count 1; the store pair; the per-node fractional spine and the whole write.

Not proved: the model-to-machine correspondence; the compiler emitting the reuse shape for arbitrary code; functions and closures.

The bottom line: if the discipline makes the actual count 1, everything else is already compiled away. The open question is whether it does — and that is the next piece.